



CHƯƠNG 4

Quản trị mã nguồn dự án với Git & CI/CD cơ bản

Triển khai và quản trị hệ thống phần
mềm



Giới thiệu nội dung

Mục tiêu chương

- Trình bày vai trò quản lý mã nguồn và các khái niệm cốt lõi của Git (repository, branch, commit).
- Thiết lập xác thực GitHub (SSH) và dùng thành thạo các lệnh Git cơ bản.
- Giải thích quy trình CI/CD và xây dựng một workflow GitHub Actions cơ bản.

Nội dung chính

- 1 Cơ bản về Git**
- 2 Thực hành Git trên máy cá nhân**
- 3 CI/CD và GitHub Actions**

Vì sao cần quản lý mã nguồn dự án?

- **Source code** là sản phẩm cốt lõi: tập lệnh mô tả toàn bộ logic và cách vận hành của một chương trình.
- Trong dự án có nhiều lập trình viên, quản lý mã nguồn (SCM/VCS) ghi nhận **ai thay đổi gì, khi nào**, giúp khôi phục khi có lỗi.
- Hỗ trợ làm việc nhóm qua branching/merging, giảm **xung đột code (merge conflict)**.
- Quản lý phiên bản/phát hành (1.0, 1.1, 2.0...) và tăng khả năng khôi phục khi mất hoặc hỏng mã nguồn.
- Là nền tảng cho các quy trình chuyên nghiệp: CI/CD, code review, pull request.

Trọng tâm chương này

Git là hệ thống quản lý mã nguồn phân tán phổ biến nhất hiện nay.

Các nền tảng quản lý mã nguồn phổ biến

- **GitHub:** nền tảng lớn nhất thế giới (thuộc Microsoft), cộng đồng hơn 100 triệu lập trình viên, mạnh Open Source, tích hợp GitHub Actions và Copilot.
- **GitLab:** nền tảng DevOps hoàn chỉnh, mạnh self-hosted, CI/CD tích hợp sẵn rất mạnh, mô hình open-core.
- **Bitbucket:** thuộc Atlassian, tích hợp sâu với Jira, phù hợp doanh nghiệp đang dùng hệ sinh thái Atlassian.
- Ngoài ra còn **Launchpad** (gắn với hệ sinh thái Ubuntu/Linux), **Codebase** và **Beanstalk** (đơn giản, mạnh về deploy) – ít phổ biến hơn.

So sánh nền tảng SCM và lựa chọn cho sinh viên CNTT

Tiêu chí	GitHub	GitLab	Bitbucket
Hosting	Cloud (+ Enterprise)	Cloud + Self-hosted mạnh	Cloud + Server
CI/CD	GitHub Actions (mạnh)	Tích hợp sẵn rất mạnh	Pipelines
Open source/Self-host	Không (proprietary)	Open-core, self-host miễn phí	Không
Phù hợp cho	Cá nhân, mã nguồn mở, doanh nghiệp	Team DevOps, doanh nghiệp lớn	Team dùng Atlassian

Khuyến nghị cho sinh viên CNTT

Bắt đầu với **GitHub** vì đây là nơi nhà tuyển dụng và cộng đồng thường xem portfolio; học **GitLab** nếu muốn hiểu sâu DevOps/CI-CD; dùng **Bitbucket** nếu trường hoặc công ty đang dùng Jira.

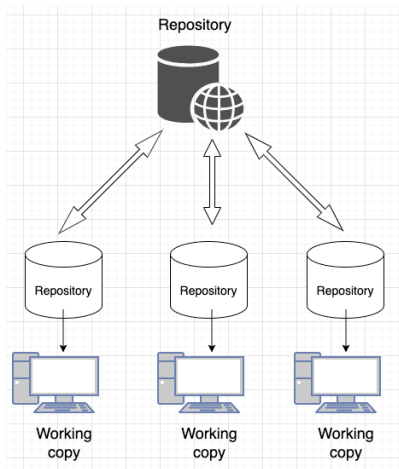
Cài đặt Git và thiết lập thông tin người dùng

- Tải bộ cài chính thức tại git-scm.com, chọn phiên bản cho Windows, macOS hoặc Linux.
- Chạy tệp cài đặt, giữ nguyên các tùy chọn mặc định (Next liên tục) để hoàn tất.
- Sau khi cài, mở **Git Bash** (Windows) hoặc **Terminal** (macOS/Linux) để thiết lập thông tin cá nhân.

```
$git config --global user.name "Tên người dùng"  
$git config --global user.email "email@example.com"  
$git config --global -l
```

Repository: kho lưu trữ mã nguồn

- **Repository (repo):** nơi chứa toàn bộ mã nguồn của dự án, dữ liệu ẩn trong thư mục `.git`.
- **Local repository:** cài đặt trên máy cá nhân của lập trình viên.
- **Remote repository:** cài đặt trên server chuyên dụng, điển hình là GitHub.
- Hai loại repo đồng bộ với nhau thông qua các lệnh Git.

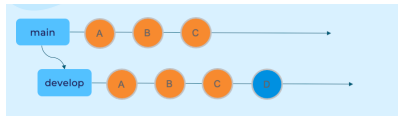


Branch: làm việc song song không xung đột

- Dự án nhiều thành viên $\Rightarrow$ nhiều task xử lý cùng lúc $\Rightarrow$ dễ **đụng code** nếu sửa chung một dòng lịch sử.
- **Branch** là nhánh riêng ghi lại luồng thay đổi độc lập; hoạt động trên một branch không ảnh hưởng các branch khác.
- Cho phép nhiều thay đổi diễn ra **đồng thời** trên cùng một repository, giải quyết nhiều nhiệm vụ song song.
- Khi tạo repository mới, Git tự động thiết lập một branch mặc định (thường gọi là **master**) chứa mọi hoạt động ban đầu.

Commit và mã định danh SHA-1

- **Commit** lưu lại một “snapshot” các thay đổi trên tập tin, kèm tác giả, thời gian và mô tả.
- Mỗi commit gắn một **mã băm SHA-1** dài 40 ký tự, duy nhất – dùng làm định danh của commit đó.
- Ví dụ: nhánh `main` có 3 commit A, B, C. Tách nhánh `develop` (cũng A, B, C), tạo thêm commit D trên `develop`.
- Kết quả: `develop` có 4 commit, trong khi `main` vẫn giữ nguyên 3 commit.



Thuật ngữ Git cần nhớ

Thuật ngữ	Ý nghĩa
HEAD	Đại diện cho commit mới nhất trong nhánh đang làm việc.
Origin	Tên mặc định Git dùng để chỉ remote repository đã kết nối.
Merge	Hợp nhất thay đổi từ nhánh này vào nhánh khác, thường tạo commit mới.
Rebase	Sắp xếp lại commit theo hướng tuyến tính để hợp nhất, không tạo commit merge.
Conflict	Xung đột giữa các thay đổi trên cùng tệp tin khi hợp nhất nhánh.
Tag	Đánh dấu một phiên bản cụ thể, thường dùng để phát hành chính thức.
.gitignore	Liệt kê các tệp Git sẽ bỏ qua, không theo dõi thay đổi.
Cherry-pick	Sao chép một commit từ nhánh khác mà không cần gộp cả nhánh.
Fork	Tạo một bản sao repository độc lập từ một nguồn khác.

Cài đặt Git trên máy cá nhân

- **Windows:** tải bộ cài từ `git-scm.com`, giữ mặc định khi cài; sau đó gọi Git từ **Git Bash** hoặc PowerShell.
- **Linux (Ubuntu):** cài trực tiếp bằng trình quản lý gói.

```
$sudo apt update  
$sudo apt install git -y  
$git --version
```

Xác nhận cài đặt thành công

Nếu terminal trả về một số phiên bản (ví dụ `git version 2.43.0`) thay vì báo lỗi “command not found”, Git đã sẵn sàng sử dụng.

Xác thực với GitHub: vì sao cần thiết?

- Clone một kho công khai **không** cần xác thực, nhưng **push** thay đổi hoặc truy cập kho riêng tư thì GitHub sẽ từ chối nếu máy chưa được nhận diện là thuộc chủ tài khoản.
- Hai cách xác thực phổ biến: **Personal Access Token (PAT)** qua giao thức HTTPS, hoặc **cặp khóa SSH** qua giao thức SSH.
- SSH được khuyến nghị cho việc học và sử dụng lâu dài: chỉ cấu hình một lần, không phải nhập lại thông tin xác thực ở mỗi lần push/pull.

Việc cần làm một lần trên mỗi máy

Cài Git và thiết lập kết nối GitHub – sau đó mới có thể thực hành các lệnh ở các slide tiếp theo.

Thiết lập kết nối SSH với GitHub

- 1 Tạo cặp khóa SSH mới:
`ssh-keygen -t ed25519 -C "email@example.com"` (lưu mặc định tại `~/.ssh/`).
- 2 Sao chép khóa công khai: `cat ~/.ssh/id_ed25519.pub`.
- 3 Trên GitHub: *Settings – SSH and GPG keys – New SSH key*, dán nội dung khóa và lưu lại.
- 4 Kiểm tra kết nối: `ssh -T git@github.com` – thành công khi thấy dòng xác nhận kèm tên tài khoản.

Lưu ý

Khóa riêng (private key) **tuyệt đối không chia sẻ**; chỉ khóa công khai (`.pub`) được đăng ký với GitHub.

Khởi tạo và cấu hình repository: init, clone, config

- `git init`: khởi tạo repository mới tại thư mục hiện tại, chạy một lần khi tạo repo.
- `git clone [url]`: tạo bản sao cục bộ của một repo từ xa, chạy một lần khi chưa có bản sao.
- `git config [--global] <thuộc tính> <giá trị>`: thiết lập thông số; `--global` áp dụng cho toàn máy, nếu không có thì chỉ áp dụng cho repo hiện tại.

```
$git init
$git clone [url]
$git config --global core.editor "code --wait"
```

Theo dõi thay đổi: status, add, rm

- `git status`: hiển thị trạng thái các tập tin (đã stage / chưa stage / untracked) và branch hiện tại.
- `git add <tập tin>`: đưa thay đổi vào Staging Area trước khi commit (`git add .` để thêm tất cả).
- `git rm <tập tin>`: xóa tập tin khỏi khu vực làm việc/staging, hoàn tất khi commit.

```
➤ ~/ITViec/example # P main +1 !1 ?1 > git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   abc.txt

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    deleted:    xyz.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    new_file.txt
```

Quản lý branch: git branch

- Tạo branch mới, xóa branch, đổi tên branch hiện tại và liệt kê các branch đang có.
- `-d` chỉ xóa được branch đã hợp nhất; dùng `-D` để **ép xóa** branch chưa hợp nhất.
- `-a` liệt kê tất cả branch (cục bộ và từ xa); `-r` chỉ liệt kê branch trên kho từ xa.

```
$git branch <tên branch mới>
$git branch -d <tên branch>
$git branch -m <tên mới>
$git branch -a
```


Chuyển nhánh: git checkout & git switch

- `git checkout <tên branch>`: chuyển trạng thái làm việc giữa các branch hoặc các phiên bản.
- `checkout -b <tên mới>`: tạo branch mới và chuyển sang ngay lập tức.
- `checkout -- <tập tin>`: khôi phục tập tin về trạng thái đã commit trước đó.
- `git switch <tên branch>` (hoặc `switch -c <tên>`): lệnh chuyên biệt chỉ để chuyển/tạo branch, tách bạch khỏi các chức năng khác của checkout.

Lưu tạm thay đổi: git stash

- `git stash`: lưu tạm các thay đổi chưa commit, làm sạch khu vực làm việc.
- `git stash apply <id>`: khôi phục một stash cụ thể, vẫn giữ lại stash đó.
- `git stash pop`: khôi phục stash mới nhất và xóa nó khỏi danh sách.
- `git stash list`: xem danh sách các stash hiện có.

```
➤ ~/ITViec/example ➤ main !2 > git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   abc.txt
        modified:   xyz.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Trước khi `git stash`

```
➤ ~/ITViec/example ➤ main !2 > git stash
Saved working directory and index state WIP on main: c0f92f5 init
➤ ~/ITViec/example ➤ main *1 > git status
On branch main
nothing to commit, working tree clean
```

Sau khi `git stash`

Lưu lại lịch sử: git commit

- `git commit -m "nội dung mô tả"`: tạo commit lưu lại thay đổi, nên có mô tả rõ ràng.
- Mỗi commit lưu lại: các thay đổi thực hiện, tác giả, thời gian và nội dung mô tả.
- `git commit -amend`: thay đổi nội dung của commit trước đó.
- `git commit -a -m "..."`: kết hợp `git add` và `git commit` trong một lệnh.

Kết nối & đẩy code lên kho từ xa: remote, push

- `git remote add origin <url>`: thêm kết nối tới kho từ xa (thường đặt tên `origin`).
- `git remote remove origin /`
`git remote rename origin <tên mới>`: xóa hoặc đổi tên bí danh remote.
- `git push origin <branch>`: đẩy các commit ở kho cục bộ lên kho từ xa.
- `git push -u origin <branch>`: thiết lập branch hiện tại theo dõi branch tương ứng trên remote ngay từ lần đầu – các lần đẩy sau không cần chỉ định lại tên branch.

Đồng bộ từ kho từ xa: git fetch & git pull

- `git fetch <remote>`: tải các thay đổi mới về nhưng **không** tự hợp nhất – an toàn để kiểm tra trước khi merge.
- `git fetch --all`: lấy toàn bộ branch, tag và commit từ kho từ xa.
- `git pull <remote> <branch>`: kết hợp giữa `fetch` và `merge` đối với nhánh đang làm việc.
- `git pull -r <remote> <branch>`: thay thế `merge` bằng `rebase` trong lúc pull.

So sánh git merge và git rebase

git merge

- Hợp nhất thay đổi từ nhánh này sang nhánh khác.
- Nếu có commit mới trên cả hai nhánh kể từ lần tách, tạo ra một **commit hợp nhất** ghi lại nội dung kết hợp.

git rebase

- Cũng hợp nhất thay đổi từ nhánh này sang nhánh hiện tại.
- Chèn các commit của nhánh được rebase lên **đầu** danh sách commit của nhánh hiện tại theo hướng tuyến tính; **không** tạo commit hợp nhất.

Câu hỏi ôn tập

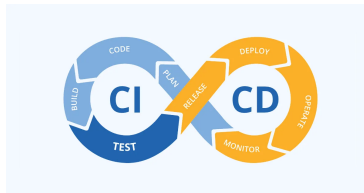
Dựa trên đặc điểm vừa nêu, hãy thử trả lời: nên ưu tiên dùng **merge** hay **rebase** trong tình huống nào?

Kiểm tra thay đổi và lịch sử: git log & git diff

- `git log`: hiển thị toàn bộ lịch sử commit theo thứ tự thời gian của repository.
- `git diff <commit 1> <commit 2>`: so sánh khác biệt giữa hai commit.
- `git diff --staged`: so sánh tập tin trong Staging Area với phiên bản đã commit trước đó.
- `git diff <branch 1> <branch 2>`: so sánh khác biệt giữa hai branch.

CI/CD là gì?

- **CI/CD** = Continuous Integration (tích hợp liên tục) + Continuous Delivery/Deployment (chuyển giao/triển khai liên tục).
- Quy trình tự động hóa **build, test và deploy** mỗi khi lập trình viên cập nhật mã nguồn, thay vì triển khai thủ công.
- Thường kết hợp cùng DevOps và Agile để sản phẩm được cải tiến liên tục, nhanh chóng đến tay người dùng.



Ưu điểm và những lưu ý khi áp dụng CI/CD

Ưu điểm

- Tự động kiểm thử, phát hiện lỗi sớm, giảm rủi ro triển khai.
- Nhóm nắm bắt thông tin kịp thời, tăng phối hợp.
- Tiết kiệm thời gian nhờ loại bỏ kiểm thử thủ công.
- Phản hồi và xử lý sự cố nhanh chóng.

Cần lưu ý

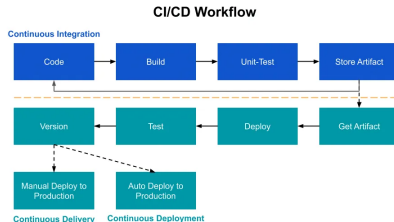
- Đòi hỏi kiến thức và kinh nghiệm vận hành.
- Hệ thống phức tạp, cần cấu hình và bảo trì thường xuyên.
- Chi phí đầu tư ban đầu cao, nhất là doanh nghiệp nhỏ.
- Một số hệ thống cũ có thể không tương thích.

Nên áp dụng sớm

CI/CD nên được áp dụng càng sớm càng tốt, kể cả với dự án nhỏ hay cá nhân; nếu đội ngũ chưa quen test tự động, có thể áp dụng **từng phần**.

Từ Continuous Integration đến Delivery/Deployment

- **Continuous Integration:**
code → build → unit test → lưu artifact.
- **Continuous Delivery:** lấy artifact → test → deploy → version, sẵn sàng phát hành nhưng **đẩy lên production thủ công**.
- **Continuous Deployment:** giống Delivery nhưng **tự động** đẩy lên production, không cần thao tác thủ công.



Quy trình CI/CD trong DevOps: hai giai đoạn

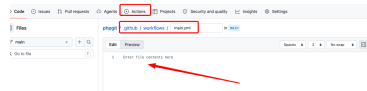
- 1 **Phát triển (Development):** tạo branch mới cho tính năng; khi hoàn tất, gửi **Pull Request** để hợp nhất vào nhánh chính.
- 2 Pipeline CI/CD **tự động kích hoạt:** build, kiểm thử, kiểm tra lint và báo cáo lỗi nếu có.
- 3 Pull Request được duyệt và hợp nhất $\Rightarrow$ chuyển sang giai đoạn **Triển khai (Deployment)**.
- 4 Mã nguồn được build thành sản phẩm, tự động đẩy lên nền tảng phân phối (ví dụ TestFlight, Firebase App Distribution).
- 5 Kết quả được thông báo qua các kênh (ví dụ Slack) để nhóm tiếp tục đánh giá.

Khó khăn khi triển khai CI/CD trong thực tế

- **Chọn công cụ phù hợp:** nhiều nền tảng (CircleCI, Bitrise, GitHub Actions...), cần thời gian thử nghiệm, đánh giá.
- **Tốn thời gian cấu hình ban đầu,** đặc biệt nếu team chưa có kinh nghiệm.
- **Thiếu kỹ năng viết unit test** và code sạch, khó kiểm thử tự động.
- **Sự kháng cự từ thành viên** khi quy trình mới gò bó, thiếu đồng thuận trong team.
- **Chi phí công cụ/hạ tầng** hạn chế ở gói miễn phí của một số công cụ.
- **Đòi hỏi liên tục cập nhật, tinh chỉnh** quy trình theo tốc độ và quy mô dự án.

GitHub Actions: CI/CD tích hợp sẵn trên GitHub

- Cho phép tự động hóa toàn bộ quy trình phát triển: xây dựng, kiểm thử, triển khai ngay trong repository.
- Hoạt động thông qua **workflow** – được kích hoạt khi có sự kiện cụ thể xảy ra trong repo.
- Workflow định nghĩa bằng tệp YAML, lưu trong thư mục `.github/workflows/`.
- Chỉ dùng được khi dự án đang trên GitHub; hiện là công cụ CI/CD được sử dụng nhiều nhất.



Thành phần GitHub Actions: Jobs và Runners

Jobs

- Tập hợp các bước (steps) cần thực hiện trong workflow.
- Mỗi job chạy trên một runner riêng biệt, có thể chạy song song hoặc tuần tự.
- **Isolation:** mỗi job chạy trong môi trường cô lập, tránh xung đột.

Runners

- Máy chủ/máy ảo (Ubuntu, Windows, macOS...) nơi job được thực thi.
- **GitHub-hosted:** do GitHub quản lý và bảo trì sẵn.
- **Self-hosted:** tự thiết lập trên máy chủ riêng, tùy chỉnh môi trường theo nhu cầu dự án.

Thành phần GitHub Actions: Actions và Events

Actions

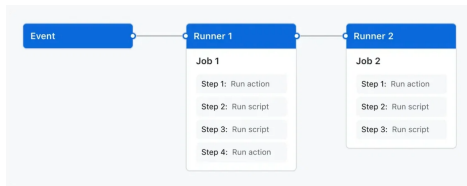
- Nhiệm vụ thực thi trong workflow – có thể là một lệnh shell hoặc phần mềm đóng gói sẵn.
- `uses`: từ khóa để dùng một action trong bước của job.
- **Marketplace**: kho action do cộng đồng và GitHub cung cấp.

Events

- Sự kiện kích hoạt workflow.
- `on`: định nghĩa sự kiện kích hoạt.
- Phổ biến: `push`, `pull_request`, `schedule`, `workflow_dispatch`.

Mối quan hệ Event – Runner – Job – Step

- Một **Event** (ví dụ **push**) kích hoạt workflow.
- Workflow phân chia công việc thành các **Job**, mỗi Job chạy trên một **Runner** riêng.
- Mỗi Job gồm nhiều **Step** tuần tự, thực thi action hoặc script.



Ví dụ: workflow `learn-github-actions.yml`

- Tạo thư mục `.github/workflows/` trong repository, sau đó thêm tệp `learn-github-actions.yml`.
- Ví dụ dưới đây cài thư viện `bats` trên môi trường Ubuntu và kiểm tra phiên bản vừa cài.

```
name: learn-github-actions
run-name: ${github.actor} is learning GitHub Actions
on: [push]
jobs:
  check-bats-version:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: npm install -g bats
      - run: bats -v
```

Giải thích các trường trong file workflow

Trường	Ý nghĩa
<code>name</code>	Tên workflow hiển thị ở tab Actions; nếu bỏ qua, dùng tên tệp thay thế.
<code>run-name</code>	Tên động, thay đổi theo mỗi lần chạy (ví dụ tên người trigger workflow).
<code>on</code>	Định nghĩa sự kiện (trigger) mà workflow sẽ chạy, ví dụ sự kiện <code>push</code> .
<code>jobs</code>	Danh sách các công việc trong workflow.
<code>runs-on</code>	Môi trường thực thi (Ubuntu, Windows Server...).
<code>steps</code>	Các bước cần thực hiện trong một job.
<code>uses</code>	Hành động (action) được sử dụng trong từng bước.

Tổng kết chương

Những điểm cần ghi nhớ

- SCM/VCS giúp theo dõi lịch sử thay đổi và hỗ trợ làm việc nhóm; GitHub là điểm khởi đầu phù hợp cho sinh viên CNTT.
- Git là hệ quản lý mã nguồn phân tán: repository, branch, commit gắn SHA-1, merge, rebase, conflict.
- Trước khi thao tác với GitHub, mỗi máy cần cài Git và thiết lập xác thực (SSH hoặc PAT) một lần.
- CI/CD tự động hóa build – test – deploy; GitHub Actions hiện thực bằng workflow YAML gồm job, runner, action, event.